vakint p-integrals: RustRed vs Forcer

clean benchmark `bench-clean` (`bench-clean.v1`) · 2026-11-11 · itphlies · pushed tip gammaloop `a3d4f5e69`, rustred `a8dcfd67` · measured build gammaloop `05cd2781d`, rustred `220ff621` (runtime-equivalent: last section)

In short. One harness, one build and one data set for every number: 630 rows (cell × backend × threads × state, medians of 1881 measurements in 92 runs, 03:21–09:45Z), 1L–4L, RustRed (M5 combined package) against Forcer, at 1 and 16 threads, cold and warm, time and peak RSS. Older benchmark rows are not mixed in.

- **Time.** RustRed is faster than Forcer in every 1L–3L cell at both thread counts. At 4L it is slower in 2 cell(s) at 1 thread (`L4_d1_r8` ×1.82, `L4_d2_r8` ×1.41) and in 3 at 16 threads (`L4_d1_r8` ×1.38, `L4_d2_r8` ×1.36, `physics_L4` ×1.42). RustRed hardly gains from 16 threads at 4L; Forcer (`tform -w16`) does, notably on `physics_L4`.
- **16-thread numbers are upper bounds.** The host was shared and loaded: on the 16 CPUs of a 16-thread job unpinned single-thread processes of other users took a median of 3.92 cores per job (median over 43 jobs; up to 8.74 cores in single samples). Both backends were exposed alike; 1-thread jobs ran on CPUs with a median foreign load of < 0.001 cores.
- **Memory.** At 1L–3L RustRed's peak RSS is below Forcer's (process plus FORM) in 93 of 94 cells × thread counts (median ×0.77; above: `L3_d1_r8` at 16 thread(s) ×1.12). At 4L it is above Forcer's in 15 of 16 cells (below: `physics_L4` ×0.59 / ×0.46): median ×3.80 at 1 thread and ×3.94 at 16 threads, worst `L4_d1_r8` (4.4 GiB against 202 MiB, ×22.5) and `L4_d1_r8` at 16 threads (10.3 GiB against 350 MiB, ×30.1).
- **Results.** All 606 rows with a pinned reference are byte-identical to it (the outputs pinned by benchmark M3 in `data/p_results.json`, re-checked when this document is built); 24 RustRed rows (`L4_d0_r6`, `L4_d0_r8`, `L4_d1_r8`, `L4_d2_r8`) have no pinned reference (M3 capped them); every cell gave one output per backend across states, thread counts and repetitions.

Table 1: RustRed over Forcer per loop count and thread count, medians over the cells (grid and physics cell) of the per-cell medians over repetitions. Cold evaluate: wall time of `Vakint::evaluate`. Cold process: the whole RustRed process (load, warm-up, evaluate, exit) without its cache save, over Forcer's whole process. Peak RSS: RustRed's process against Forcer's process plus its FORM subprocess. Slower: cold evaluate ratio > 1.

L	thr.	cells	cold evaluate		cold process		peak RSS		slower than Forcer
			median	worst	median	worst	median	worst	
1 L	1	15	×1/100	×0.03 (L1 d1 r8)	×0.17	×0.17	×0.77	×0.77 (L1 d0 r4)	none
1 L	16	15	×0.02	×0.08 (L1 d1 r8)	×0.17	×0.19	×0.75	×0.77 (L1 d0 r2)	none
2 L	1	16	×0.01	×0.04 (physics L2)	×0.16	×0.17	×0.77	×0.77 (physics L2)	none
2 L	16	16	×0.02	×0.10 (physics L2)	×0.16	×0.19	×0.75	×0.78 (L2 d0 r0)	none
3 L	1	16	×0.02	×0.32 (physics L3)	×0.09	×0.32	×0.77	×0.77 (L3 d2 r4)	none
3 L	16	16	×0.02	×0.46 (physics L3)	×0.09	×0.41	×0.75	×1.12 (L3 d1 r8)	none
4 L	1	16	×0.09	×1.82 (L4 d1 r8)	×0.11	×1.83	×3.80	×22.5 (L4 d1 r8)	**L4 d1 r8, L4 d2 r8**
4 L	16	16	×0.07	×1.42 (physics L4)	×0.09	×1.42	×3.94	×30.1 (L4 d1 r8)	**L4 d1 r8, L4 d2 r8, physics L4**



Reading guide (Tables 2 and 3). Each cell sums all tops of that loop count at the given dots, times a numerator of that rank. Bold: RustRed cold (seconds, median of 3). F: Forcer cold; ×: RustRed cold over Forcer cold (colour); ws: RustRed warm, a fresh process on its own cold cache; wp: the warm repeat inside the cold process; Fw: Forcer warm (materialised). RSS: RustRed's cold peak RSS and its ratio to Forcer's (red: RustRed higher). ✓: result exactly equal to Forcer's (M3 exact tier, `p_exact.json`, carried by the byte-identical outputs); n.c.: not exact-checked (cost). †: two repetitions instead of three.

Table 2: 1 thread (`n_cores` 1, `form`); RustRed arithmetic `modular_polynomial`.

L	dots	rank 0	rank 2	rank 4	rank 6	rank 8
1 L	0	**0.004** ✓ F 0.417 **×1/110** ws 0.004 wp 0.003 Fw 0.418 RSS 96 MiB ×0.77	**0.004** ✓ F 0.418 **×1/104** ws 0.004 wp 0.003 Fw 0.416 RSS 96 MiB ×0.77	**0.004** ✓ F 0.421 **×1/101** ws 0.004 wp 0.004 Fw 0.422 RSS 96 MiB ×0.77	**0.004** ✓ F 0.435 **×1/100** ws 0.004 wp 0.004 Fw 0.423 RSS 96 MiB ×0.77	**0.004** ✓ F 0.425 **×0.01** ws 0.004 wp 0.004 Fw 0.425 RSS 96 MiB ×0.77
	1	**0.004** ✓ F 0.418 **×1/109** ws 0.004 wp 0.003 Fw 0.418 RSS 96 MiB ×0.77	**0.005** ✓ F 0.420 **×0.01** ws 0.005 wp 0.004 Fw 0.417 RSS 96 MiB ×0.77	**0.007** ✓ F 0.423 **×0.02** ws 0.006 wp 0.006 Fw 0.426 RSS 96 MiB ×0.77	**0.008** ✓ F 0.423 **×0.02** ws 0.008 wp 0.008 Fw 0.427 RSS 96 MiB ×0.77	**0.011** ✓ F 0.430 **×0.03** ws 0.010 wp 0.010 Fw 0.429 RSS 96 MiB ×0.77
	2	**0.004** ✓ F 0.415 **×1/106** ws 0.004 wp 0.004 Fw 0.419 RSS 96 MiB ×0.77	**0.004** ✓ F 0.422 **×1/106** ws 0.004 wp 0.003 Fw 0.422 RSS 96 MiB ×0.77	**0.004** ✓ F 0.422 **×1/101** ws 0.004 wp 0.004 Fw 0.420 RSS 96 MiB ×0.77	**0.004** ✓ F 0.421 **×0.01** ws 0.004 wp 0.004 Fw 0.418 RSS 96 MiB ×0.77	**0.004** ✓ F 0.427 **×0.01** ws 0.004 wp 0.004 Fw 0.425 RSS 96 MiB ×0.77
2 L	0	**0.005** ✓ F 0.450 **×0.01** ws 0.005 wp 0.004 Fw 0.455 RSS 96 MiB ×0.77	**0.005** ✓ F 0.450 **×0.01** ws 0.005 wp 0.004 Fw 0.451 RSS 96 MiB ×0.76	**0.006** ✓ F 0.449 **×0.01** ws 0.005 wp 0.004 Fw 0.449 RSS 96 MiB ×0.76	**0.009** ✓ F 0.459 **×0.02** ws 0.006 wp 0.005 Fw 0.459 RSS 96 MiB ×0.76	**0.011** ✓ F 0.471 **×0.02** ws 0.007 wp 0.006 Fw 0.473 RSS 96 MiB ×0.77
	1	**0.005** ✓ F 0.451 **×0.01** ws 0.005 wp 0.004 Fw 0.452 RSS 96 MiB ×0.77	**0.006** ✓ F 0.461 **×0.01** ws 0.006 wp 0.004 Fw 0.459 RSS 96 MiB ×0.76	**0.005** ✓ F 0.458 **×0.01** ws 0.005 wp 0.004 Fw 0.458 RSS 96 MiB ×0.76	**0.009** ✓ F 0.455 **×0.02** ws 0.006 wp 0.005 Fw 0.456 RSS 96 MiB ×0.76	**0.012** ✓ F 0.469 **×0.02** ws 0.007 wp 0.006 Fw 0.468 RSS 96 MiB ×0.77
	2	**0.006** ✓ F 0.456 **×0.01** ws 0.006 wp 0.004 Fw 0.451 RSS 96 MiB ×0.76	**0.006** ✓ F 0.459 **×0.01** ws 0.006 wp 0.005 Fw 0.457 RSS 96 MiB ×0.77	**0.007** ✓ F 0.458 **×0.01** ws 0.006 wp 0.005 Fw 0.458 RSS 96 MiB ×0.77	**0.009** ✓ F 0.467 **×0.02** ws 0.007 wp 0.006 Fw 0.466 RSS 96 MiB ×0.77	**0.012** ✓ F 0.482 **×0.02** ws 0.009 wp 0.008 Fw 0.485 RSS 96 MiB ×0.77
3 L	0	**0.027** ✓ F 1.63 **×0.02** ws 0.025 wp 0.008 Fw 1.63 RSS 96 MiB ×0.77	**0.041** ✓ F 1.72 **×0.02** ws 0.030 wp 0.013 Fw 1.73 RSS 96 MiB ×0.77	**0.029** ✓ F 1.98 **×0.01** ws 0.021 wp 0.011 Fw 1.98 RSS 96 MiB ×0.77	**0.138** ✓ F 2.37 **×0.06** ws 0.068 wp 0.056 Fw 2.35 RSS 96 MiB ×0.77	**0.376** ✓ F 2.75 **×0.14** ws 0.179 wp 0.165 Fw 2.76 RSS 96 MiB ×0.77
	1	**0.030** ✓ F 2.49 **×0.01** ws 0.027 wp 0.010 Fw 2.36 RSS 96 MiB ×0.77	**0.042** ✓ F 2.38 **×0.02** ws 0.032 wp 0.015 Fw 1.82 RSS 96 MiB ×0.77	**0.045** ✓ F 2.49 **×0.02** ws 0.033 wp 0.016 Fw 2.33 RSS 96 MiB ×0.77	**0.226** ✓ F 3.11 **×0.07** ws 0.120 wp 0.100 Fw 3.07 RSS 96 MiB ×0.77	**0.724** ✓ F 3.33 **×0.22** ws 0.369 wp 0.357 Fw 3.46 RSS 96 MiB ×0.77
	2	**0.035** ✓ F 1.92 **×0.02** ws 0.030 wp 0.013 Fw 1.92 RSS 96 MiB ×0.77	**0.047** ✓ F 2.03 **×0.02** ws 0.039 wp 0.019 Fw 2.03 RSS 96 MiB ×0.77	**0.049** ✓ F 2.17 **×0.02** ws 0.038 wp 0.019 Fw 2.16 RSS 96 MiB ×0.77	**0.191** ✓ F 2.56 **×0.07** ws 0.113 wp 0.095 Fw 2.55 RSS 96 MiB ×0.77	**0.563** ✓ F 3.25 **×0.17** ws 0.325 wp 0.316 Fw 3.25 RSS 96 MiB ×0.77
4 L	0	**1.37** ✓ F 19.7 **×0.07** ws 1.10 wp 0.239 Fw 19.9 RSS 314 MiB ×2.49	**2.26** ✓ F 61.1 **×0.04** ws 1.55 wp 0.580 Fw 61.9 RSS 406 MiB ×3.15	**2.42** n.c. F 44.8 **×0.05** ws 1.59 wp 0.786 Fw 45.2 RSS 460 MiB ×3.65	**34.0** n.c. F 82.5 **×0.41** ws 19.4 wp 20.4 Fw 84.4 RSS 1.5 GiB ×8.90	**103** n.c. F 173 **×0.59** ws 73.3 wp 76.9 Fw 196 RSS 2.1 GiB ×10.2
	1	**2.40** ✓ F 34.7 **×0.07** ws 1.85 wp 0.945 Fw 34.3 RSS 376 MiB ×2.95	**3.70** ✓ F 41.6 **×0.09** ws 2.87 wp 1.92 Fw 41.4 RSS 445 MiB ×3.29	**3.00** n.c. F 48.3 **×0.06** ws 2.11 wp 1.13 Fw 48.4 RSS 500 MiB ×3.87	**50.1** n.c. F 116 **×0.43** ws 36.7 wp 32.3 Fw 160 RSS 1.9 GiB ×10.7	**329** n.c. F 181 **×1.82** ws 232 wp 233 Fw 170 RSS 4.4 GiB ×22.5
	2	**4.16** ✓ F 41.8 **×0.10** ws 3.17 wp 2.32 Fw 41.7 RSS 430 MiB ×2.83	**4.83** ✓ F 54.9 **×0.09** ws 3.55 wp 2.67 Fw 47.0 RSS 493 MiB ×3.73	**3.94** n.c. F 56.2 **×0.07** ws 2.67 wp 1.69 Fw 57.5 RSS 529 MiB ×4.10	**95.7** n.c. F 131 **×0.73** ws 51.8 wp 62.5 Fw 120 RSS 1.9 GiB ×10.3	**394** n.c. F 280 **×1.41** ws 291 wp 286 Fw 260 RSS 4.7 GiB ×18.3
phys.		2 L **0.020** ✓ F 0.472 **×0.04** ws 0.019 wp 0.018 Fw 0.479 RSS 96 MiB ×0.77	3 L **0.427** ✓ F 1.33 **×0.32** ws 0.409 wp 0.399 Fw 1.32 RSS 96 MiB ×0.77	4 L **80.6** ✓ F 143 **×0.56** ws 83.7 wp 80.2 Fw 121 RSS 341 MiB ×0.59	5L: not part of this benchmark	

Table 3: 16 threads (`n_cores` 16, `tform -w16`) on 16 CPUs of one NUMA node; upper bounds (foreign load, Table 1 note); RustRed arithmetic `modular_polynomial`.

L	dots	rank 0	rank 2	rank 4	rank 6	rank 8
1 L	0	**0.007** ✓ F 0.430 **×0.02** ws 0.006 wp 0.006 Fw 0.429 RSS 96 MiB ×0.75	**0.007** ✓ F 0.435 **×0.02** ws 0.006 wp 0.006 Fw 0.430 RSS 96 MiB ×0.77	**0.007** ✓ F 0.441 **×0.02** ws 0.006 wp 0.006 Fw 0.439 RSS 96 MiB ×0.75	**0.007** ✓ F 0.440 **×0.02** ws 0.006 wp 0.006 Fw 0.441 RSS 96 MiB ×0.75	**0.008** ✓ F 0.445 **×0.02** ws 0.006 wp 0.006 Fw 0.443 RSS 96 MiB ×0.77
	1	**0.007** ✓ F 0.436 **×0.02** ws 0.006 wp 0.006 Fw 0.432 RSS 96 MiB ×0.77	**0.014** ✓ F 0.441 **×0.03** ws 0.012 wp 0.011 Fw 0.442 RSS 96 MiB ×0.75	**0.022** ✓ F 0.448 **×0.05** ws 0.018 wp 0.018 Fw 0.444 RSS 96 MiB ×0.77	**0.030** ✓ F 0.449 **×0.07** ws 0.024 wp 0.024 Fw 0.448 RSS 96 MiB ×0.75	**0.039** ✓ F 0.456 **×0.08** ws 0.031 wp 0.030 Fw 0.464 RSS 96 MiB ×0.75
	2	**0.007** ✓ F 0.439 **×0.02** ws 0.006 wp 0.006 Fw 0.432 RSS 96 MiB ×0.77	**0.007** ✓ F 0.435 **×0.02** ws 0.006 wp 0.006 Fw 0.447 RSS 96 MiB ×0.77	**0.007** ✓ F 0.447 **×0.02** ws 0.006 wp 0.007 Fw 0.437 RSS 96 MiB ×0.77	**0.008** ✓ F 0.441 **×0.02** ws 0.006 wp 0.006 Fw 0.436 RSS 96 MiB ×0.75	**0.008** ✓ F 0.439 **×0.02** ws 0.007 wp 0.006 Fw 0.437 RSS 96 MiB ×0.75
2 L	0	**0.008** ✓ F 0.475 **×0.02** ws 0.008 wp 0.007 Fw 0.478 RSS 96 MiB ×0.78	**0.009** ✓ F 0.471 **×0.02** ws 0.007 wp 0.008 Fw 0.475 RSS 96 MiB ×0.75	**0.010** ✓ F 0.474 **×0.02** ws 0.007 wp 0.007 Fw 0.484 RSS 96 MiB ×0.75	**0.013** ✓ F 0.488 **×0.03** ws 0.009 wp 0.008 Fw 0.493 RSS 96 MiB ×0.78	**0.016** ✓ F 0.498 **×0.03** ws 0.009 wp 0.009 Fw 0.498 RSS 96 MiB ×0.77
	1	**0.008** ✓ F 0.476 **×0.02** ws 0.008 wp 0.007 Fw 0.488 RSS 96 MiB ×0.75	**0.010** ✓ F 0.488 **×0.02** ws 0.009 wp 0.007 Fw 0.494 RSS 96 MiB ×0.75	**0.009** ✓ F 0.488 **×0.02** ws 0.008 wp 0.007 Fw 0.490 RSS 96 MiB ×0.76	**0.013** ✓ F 0.482 **×0.03** ws 0.008 wp 0.008 Fw 0.479 RSS 96 MiB ×0.76	**0.016** ✓ F 0.550 **×0.03** ws 0.009 wp 0.008 Fw 0.491 RSS 96 MiB ×0.74
	2	**0.009** ✓ F 0.479 **×0.02** ws 0.009 wp 0.007 Fw 0.476 RSS 96 MiB ×0.75	**0.010** ✓ F 0.489 **×0.02** ws 0.009 wp 0.008 Fw 0.497 RSS 96 MiB ×0.76	**0.019** ✓ F 0.485 **×0.04** ws 0.010 wp 0.015 Fw 0.493 RSS 96 MiB ×0.75	**0.013** ✓ F 0.497 **×0.03** ws 0.011 wp 0.011 Fw 0.509 RSS 96 MiB ×0.74	**0.016** ✓ F 0.507 **×0.03** ws 0.013 wp 0.012 Fw 0.510 RSS 96 MiB ×0.76
3 L	0	**0.037** ✓ F 1.80 **×0.02** ws 0.040 wp 0.012 Fw 1.80 RSS 96 MiB ×0.77	**0.049** ✓ F 1.98 **×0.02** ws 0.041 wp 0.022 Fw 1.94 RSS 96 MiB ×0.73	**0.031** ✓ F 2.23 **×0.01** ws 0.026 wp 0.016 Fw 2.16 RSS 96 MiB ×0.75	**0.142** ✓ F 2.62 **×0.05** ws 0.086 wp 0.075 Fw 2.80 RSS 96 MiB ×0.65	**0.336** ✓ F 3.58 **×0.09** ws 0.224 wp 0.213 Fw 3.19 RSS 123 MiB ×0.84
	1	**0.033** ✓ F 2.38 **×0.01** ws 0.031 wp 0.014 Fw 2.53 RSS 96 MiB ×0.77	**0.045** ✓ F 2.42 **×0.02** ws 0.040 wp 0.022 Fw 2.14 RSS 96 MiB ×0.76	**0.047** ✓ F 2.32 **×0.02** ws 0.042 wp 0.023 Fw 2.41 RSS 96 MiB ×0.76	**0.212** ✓ F 2.79 **×0.08** ws 0.157 wp 0.139 Fw 2.78 RSS 96 MiB ×0.71	**0.638** ✓ F 3.16 **×0.20** ws 0.408 wp 0.384 Fw 3.18 RSS 158 MiB ×1.12
	2	**0.038** ✓ F 2.13 **×0.02** ws 0.036 wp 0.018 Fw 2.13 RSS 96 MiB ×0.73	**0.054** ✓ F 2.21 **×0.02** ws 0.050 wp 0.029 Fw 2.21 RSS 96 MiB ×0.71	**0.054** ✓ F 2.41 **×0.02** ws 0.048 wp 0.029 Fw 2.36 RSS 96 MiB ×0.75	**0.196** ✓ F 2.64 **×0.07** ws 0.155 wp 0.139 Fw 2.66 RSS 96 MiB ×0.67	**0.537** ✓ F 3.07 **×0.17** ws 0.400 wp 0.380 Fw 3.15 RSS 129 MiB ×0.86
4 L	0	**1.34** ✓ F 22.4 **×0.06** ws 1.18 wp 0.277 Fw 22.5 RSS 346 MiB ×2.26	**1.99** ✓ F 39.8 **×0.05** ws 1.71 wp 0.665 Fw 40.2 RSS 495 MiB ×2.79	**1.98** n.c. F 48.7 **×0.04** ws 1.66 wp 0.803 Fw 49.0 RSS 569 MiB ×3.65	**25.8** n.c. F 70.3 **×0.37** ws 16.4 wp 16.4 Fw 75.6 RSS 2.8 GiB ×10.2	**43.3†** n.c. F 99.0 **×0.44** ws 27.8† wp 29.3† Fw 98.1 RSS 4.1 GiB ×11.7
	1	**2.07** ✓ F 37.7 **×0.05** ws 1.81 wp 0.842 Fw 37.9 RSS 500 MiB ×2.94	**2.93** ✓ F 42.9 **×0.07** ws 2.63 wp 1.57 Fw 43.1 RSS 632 MiB ×3.34	**2.66** n.c. F 52.6 **×0.05** ws 2.27 wp 1.17 Fw 55.2 RSS 625 MiB ×3.66	**34.1** n.c. F 80.1 **×0.43** ws 25.4 wp 24.9 Fw 84.1 RSS 3.5 GiB ×12.5	**135†** n.c. F 98.2 **×1.38** ws 86.6† wp 92.8† Fw 100 RSS 10.3 GiB ×30.1
	2	**3.12** ✓ F 41.7 **×0.07** ws 2.67 wp 1.80 Fw 42.9 RSS 713 MiB ×3.06	**3.55** ✓ F 50.4 **×0.07** ws 3.01 wp 2.06 Fw 51.3 RSS 790 MiB ×4.32	**3.30** n.c. F 61.8 **×0.05** ws 2.85 wp 1.71 Fw 58.0 RSS 731 MiB ×4.22	**41.3** n.c. F 94.1 **×0.44** ws 27.0 wp 27.5 Fw 91.2 RSS 3.7 GiB ×12.3	**157†** n.c. F 115 **×1.36** ws 100† wp 101† Fw 118 RSS 10.7 GiB ×23.8
phys.		2 L **0.049** ✓ F 0.507 **×0.10** ws 0.045 wp 0.043 Fw 0.508 RSS 96 MiB ×0.74	3 L **0.710** ✓ F 1.55 **×0.46** ws 0.705 wp 0.680 Fw 1.40 RSS 96 MiB ×0.70	4 L **86.5** ✓ F 61.1 **×1.42** ws 84.3 wp 85.3 Fw 68.2 RSS 536 MiB ×0.46	5L: not part of this benchmark	

Table 4: 4L in detail, cold runs, medians. RustRed: evaluate (the timed call), its reduction span, the load of engine and library before the clock, the cache save after the clock, and the whole process without that save. Forcer: evaluate and its whole process. Foreign cores: median load of other users' processes on the job's CPUs (16 threads only; 1-thread jobs had none to speak of).

cell	thr.	RustRed cold (s)					Forcer cold (s)		RustRed / Forcer		peak RSS			foreign cores RR / F
		evaluate	reduc-tion	load	save	process − save	evaluate	process	evaluate	process	RustRed	Forcer	ratio	
L4 d0 r0	1	**1.37**	1.33	0.617	0.071	2.02	**19.7**	20.2	**×0.07**	×0.10	314 MiB	126 MiB	×2.49	—
L4 d0 r0	16	**1.34**	1.31	0.635	0.073	2.03	**22.4**	22.9	**×0.06**	×0.09	346 MiB	153 MiB	×2.26	3.0 / 3.6
L4 d0 r2	1	**2.26**	2.22	0.729	0.138	3.07	**61.1**	62.0	**×0.04**	×0.05	406 MiB	129 MiB	×3.15	—
L4 d0 r2	16	**1.99**	1.95	0.674	0.137	2.76	**39.8**	40.4	**×0.05**	×0.07	495 MiB	177 MiB	×2.79	4.3 / 4.1
L4 d0 r4	1	**2.42**	2.37	0.632	0.184	3.18	**44.8**	45.3	**×0.05**	×0.07	460 MiB	126 MiB	×3.65	—
L4 d0 r4	16	**1.98**	1.94	0.643	0.177	2.81	**48.7**	49.2	**×0.04**	×0.06	569 MiB	156 MiB	×3.65	4.1 / 4.0
L4 d0 r6	1	**34.0**	33.9	0.682	2.25	35.0	**82.5**	83.1	**×0.41**	×0.42	1.5 GiB	177 MiB	×8.90	—
L4 d0 r6	16	**25.8**	25.7	0.724	2.31	27.1	**70.3**	70.9	**×0.37**	×0.38	2.8 GiB	285 MiB	×10.2	3.8 / 3.9
L4 d0 r8	1	**103**	103	1.12	5.82	105	**173**	174	**×0.59**	×0.60	2.1 GiB	210 MiB	×10.2	—
L4 d0 r8	16	**43.3†**	43.1	0.746	3.56	44.8	**99.0**	99.6	**×0.44**	×0.45	4.1 GiB	362 MiB	×11.7	3.0 / 4.0
L4 d1 r0	1	**2.40**	2.36	0.691	0.188	3.17	**34.7**	35.2	**×0.07**	×0.09	376 MiB	127 MiB	×2.95	—
L4 d1 r0	16	**2.07**	2.02	0.633	0.179	2.78	**37.7**	38.3	**×0.05**	×0.07	500 MiB	170 MiB	×2.94	3.9 / 4.0
L4 d1 r2	1	**3.70**	3.63	0.672	0.267	4.47	**41.6**	42.2	**×0.09**	×0.11	445 MiB	135 MiB	×3.29	—
L4 d1 r2	16	**2.93**	2.86	0.703	0.245	3.68	**42.9**	43.6	**×0.07**	×0.08	632 MiB	189 MiB	×3.34	3.0 / 3.0
L4 d1 r4	1	**3.00**	2.93	0.626	0.211	3.71	**48.3**	48.8	**×0.06**	×0.08	500 MiB	129 MiB	×3.87	—
L4 d1 r4	16	**2.66**	2.59	0.697	0.217	3.49	**52.6**	53.9	**×0.05**	×0.06	625 MiB	171 MiB	×3.66	6.1 / 5.2
L4 d1 r6	1	**50.1**	49.8	0.794	3.29	51.7	**116**	117	**×0.43**	×0.44	1.9 GiB	177 MiB	×10.7	—
L4 d1 r6	16	**34.1**	33.8	0.741	3.37	35.7	**80.1**	80.7	**×0.43**	×0.44	3.5 GiB	288 MiB	×12.5	4.8 / 6.5
L4 d1 r8	1	**329**	326	1.15	11.6	332	**181**	181	**×1.82**	×1.83	4.4 GiB	202 MiB	×22.5	—
L4 d1 r8	16	**135†**	134	0.698	9.56	137	**98.2**	98.7	**×1.38**	×1.39	10.3 GiB	350 MiB	×30.1	2.3 / 4.2
L4 d2 r0	1	**4.16**	4.10	0.726	0.315	4.99	**41.8**	42.3	**×0.10**	×0.12	430 MiB	152 MiB	×2.83	—
L4 d2 r0	16	**3.12**	3.06	0.636	0.313	4.02	**41.7**	42.2	**×0.07**	×0.10	713 MiB	233 MiB	×3.06	4.0 / 4.0
L4 d2 r2	1	**4.83**	4.77	0.726	0.375	5.69	**54.9**	55.7	**×0.09**	×0.10	493 MiB	132 MiB	×3.73	—
L4 d2 r2	16	**3.55**	3.49	0.630	0.363	4.32	**50.4**	51.0	**×0.07**	×0.08	790 MiB	183 MiB	×4.32	3.0 / 3.5
L4 d2 r4	1	**3.94**	3.86	0.635	0.304	4.69	**56.2**	56.7	**×0.07**	×0.08	529 MiB	129 MiB	×4.10	—
L4 d2 r4	16	**3.30**	3.23	0.656	0.313	4.09	**61.8**	62.3	**×0.05**	×0.07	731 MiB	173 MiB	×4.22	6.1 / 3.9
L4 d2 r6	1	**95.7**	95.4	0.872	5.60	97.6	**131**	131	**×0.73**	×0.74	1.9 GiB	189 MiB	×10.3	—
L4 d2 r6	16	**41.3**	41.2	0.756	3.30	42.9	**94.1**	94.7	**×0.44**	×0.45	3.7 GiB	309 MiB	×12.3	5.0 / 4.8
L4 d2 r8	1	**394**	394	1.01	13.3	398	**280**	281	**×1.41**	×1.41	4.7 GiB	261 MiB	×18.3	—
L4 d2 r8	16	**157†**	156	0.675	10.4	159	**115**	115	**×1.36**	×1.38	10.7 GiB	462 MiB	×23.8	2.9 / 3.9
physics L4	1	**80.6**	26.7	0.626	0.214	81.5	**143**	143	**×0.56**	×0.57	341 MiB	582 MiB	×0.59	—
physics L4	16	**86.5**	32.3	0.754	0.232	87.5	**61.1**	61.7	**×1.42**	×1.42	536 MiB	1.1 GiB	×0.46	3.1 / 4.0

Measured build and pushed tip

The rows were measured with one frozen harness binary (sha256 `79ee9255ffa2e5a0`…) built from gammaloop `05cd2781d` (local branch, never pushed) pinning rustred `220ff621`. They stand for the pushed tip — gammaloop `a3d4f5e69`, rustred `a8dcfd67`, vakint-external-data `85f2818f` — because the two builds are runtime-equivalent for every timed call (path by path in `data/p_receipt.json`: 5 gammaloop paths, 10 rustred source files):

1. **gammaloop** `05cd2781d`..`a3d4f5e69` (4 commits): outside the rustred pin in `Cargo.toml`/`Cargo.lock`, only test code changes (`p_data.rs` inside `mod tests`, `tests/p_integral_rustred_tests.rs`, `tests/p_library_shipping_tests.rs`); the built-in library `crates/vakint/data/p` (the 1L–3L program) is the same git tree.
2. **rustred** `220ff621`..`a8dcfd67` (5 commits): test files and goldens; the package comparison `p_library_differences` (used by `rustred p package` and tests, never on an evaluation path); `rustfmt` of the scheme lane (token-identical up to added braces); `clippy` attributes and two needless borrows. One change is reachable at run time: the rational-constant fast path of `invert_symbolic_matrix` (`family/exact.rs`) is now also conditional on diagnostics being off and on every entry being a coefficient of the context. Ordinary execution has diagnostics off (the default) and only such coefficients, so it takes the same path; the added cost is one membership test per matrix entry.
3. **4L library**: `p_integral/p-library-1-4` of vakint-external-data `85f2818f` is the measured package (`94976a1862360d60`…): all 472 files byte-identical by sha256. The 1L–3L library is the built-in one, identical in both builds (identity ends `P_ORDERING_VERSION=1:generation`).
4. The pushed tip passed the vakint gate (1097/1097 tests, R* 60/60) and the Forcer cross-check of all 22 tops of L4 d0_r8/d1_r8 through vakint; these verify results, not timings.
5. **Spot re-time**: the harness of the pushed tip (stamped release build of gammaloop a3d4f5e69 / rustred a8dcfd67, binary sha256 e440424254d5c6bb9479dc2961209314ffddaef5e7db8b58d47536b4dcbdd4c6) against the measured binary, RustRed, 1 thread, 3 repetitions, median evaluate seconds, 2026-11-11 11:07-11:16Z, same host. `physics_L3` cold 0.430 s against 0.436 s (×0.99); `L4_d0_r4` cold 2.38 s against 2.45 s (×0.97); `L4_d0_r4` warm-same 1.56 s against 1.63 s (×0.96); `L4_d1_r6` cold 46.5 s against 46.0 s (×1.01); `L4_d1_r6` warm-same 26.7 s against 27.6 s (×0.97). Outputs byte-identical between the two binaries and to the data set's rows in every cell and state above.

Method. evaluate = Vakint::evaluate wall after the load and one warm-up bubble; the RustRed cache save (fsync) runs after the clock (save_seconds), then the in-process warm repeat; process = whole child process minus that repeat (load + warm-up + evaluate + save + exit); process_excl_save = process - save. Work directory /dev/shm (tmpfs) whole harness under taskset (1 CPU at 1 thread, 16 CPUs of one NUMA node at 16 threads) and numactl --membind=<node>; 1-thread jobs in parallel on separate CPUs, 16-thread jobs one at a time. States: RustRed cold (empty ordered-cache directory, shipped caches off), warm-same (a fresh process on a read-only copy of its cold cache), warm-process (in the cold process, after the save); Forcer cold (private empty materialisation directory) and warm (materialised); Forcer has no result cache. Three repetitions per state (†: two), median; cap 3600 s per child (no row was capped or skipped). OS pages are not flushed. Physics row: massless-quark (L−1)-rung gluon self-energy, Feynman gauge, transverse projector, external momentum kept.

Provenance. `data/p_clean.json` = artefact `bench-clean`, written `2026-11-11T09:46:17Z`, copied verbatim; host itphlies (AMD EPYC 9754 128-Core Processor), `FORM 5.0.0`, Forcer `68a30bf`; RustRed at 1L–3L with the built-in `p-library-1-3`, at 4L with `p-library-1-4` listed by external data. Outputs compared with the pinned M3 outputs: gammaloop fd8b44641 crates/vakint/docs/benchmark/data/p_results.json (output_sha256 per cell and backend). Exact tier: `data/p_exact.json` (gammaloop `1a8862a97`, rustred `b7fd9f2b`, 54 cells agree); of the 4L cells it marks n.c., the gate cross-check above covers the 22 tops of L4 d0_r8/d1_r8. The M3 benchmark rows remain in `data/p_results.json` for their pinned outputs only; none of their timings is used here.